

- [CPSC 375: High-Performance Computing](#)

[CPSC 375: High-Performance Computing](#)

Spring 2026

- [Syllabus](#)
- [Schedule](#)
- [Upload](#)
- [The Cluster Project](#)

Assignment 7

Due 11:59 p.m., ~~Monday, April 13~~, Tuesday, April 14, 2026

IMPORTANT! This is an individual assignment. You may discuss broad issues of interpretation, understanding, and general approaches to a solution. However, the development of a specific solution or program code must be your own work. The assignment is expected to be entirely your own, designed and coded by you alone. If you need assistance, please consult your instructor or the TAs. Be sure to read the specific policies outlined in the [Academic Honesty in Computing](#) section.

Overview

This assignment explores two distinct approaches to parallelizing the Sieve of Eratosthenes: incremental optimization of a *domain decomposition* model and the implementation of a *functional decomposition* model. You will benchmark these versions on the Pine cluster to analyze performance gains.

Part I: Incremental Domain Decomposition Optimizations

Starting with the baseline parallel sieve program we discussed in class, implement three progressive improvements:

Eliminate Even Integers. Modify the sieve so that only odd integers are represented in the boolean array. This should halve the memory requirement and approximately double the speed of marking multiples.

Eliminate Broadcasts. In addition to the first improvement, each process should independently find all primes between 3 and $\lfloor \sqrt{n} \rfloor$ using a sequential sieve on a separate local array. This eliminates the need for MPI_Bcast calls, reducing communication overhead.

Reorganize Loops for Cache Efficiency. Reorganize the loops to improve the cache hit rate. Strike all multiples of all primes less than $\lfloor \sqrt{n} \rfloor$ on a small section of the subarray that fits in the cache before moving to the next section.

Part II: Functional Decomposition of the Parallel Sieve

Unlike domain decomposition, a functional decomposition assigns different tasks (prime factors) to different processes, even though they may operate on the same range of numbers. To find primes up to n using p processes:

1. Each process independently identifies all primes up to $\sqrt{n}$.
2. Each process sieves a full-sized local array (representing 2 to n) using only $1/p$ of the primes found in Step 1.
3. Use `MPI_Reduce` with an OR operator to merge all distributed arrays into a single array on Process 0.
4. Process 0 counts the unmarked elements to determine the total number of primes.

Benchmarking and Performance Analysis. For both parts, benchmark your programs using $n = 100,000,000$ on 1, 2, and 4 processor:

- Compare the performance of the three optimized domain decomposition versions against the baseline.
- Compare the domain decomposition model against the functional decomposition model.
- Discuss the memory constraints of functional decomposition on the Pine cluster's nodes. Since each of the p processes must maintain a full copy of the sieve array, analyze how this redundant allocation affects scalability when running multiple tasks per node.
- Explain the communication overhead introduced by the `MPI_Reduce` step in the functional model compared to the optimized domain decomposition versions.

Notes on Running MPI Jobs Using Slurm. Slurm handles resource allocation and placement, while MPI handles communication. Slurm schedules tasks (processes or ranks). Good performance comes from making those two cooperate cleanly rather than fighting each other. When running MPI jobs in a Slurm batch script, keep resource requests explicit so Slurm and MPI agree on placement and execution. Here are some suggestions:

- Specify total tasks: `#SBATCH --ntasks=N`
- Control distribution: `--nodes, --ntasks-per-node`
- Ensure: $\text{ntasks} \leq \text{nodes} \times \text{ntasks-per-node}$
- Match hardware: typically 1 rank per core
- Avoid oversubscription
- Launch with Slurm: `srun ./mpi_program`
- Avoid mixing `mpirun` unless required.

Cluster Usage Policy. To ensure the cluster remains a shared resource, resource limits are now in effect. All jobs are now capped at a maximum of 4 nodes and a 30-minute runtime. These guardrails are in place to keep the queue moving and prevent zombie processes or infinite loops from stalling the cluster's node pool.

Handin

Submit your source code and a brief report in PDF format, including your benchmarking table and performance analysis.

- **Welcome: Sean**

- [Logout](#)

